

Competitive Programming

Section 48 Editorial: Divide And Conquer, Deque, And Knuth DP

Competitive Programming Course

2026-07-18

Table of contents

1	Editorial overview	3
1.1	Problem-specific proof map	3
1.2	Detailed solution guide	3
2	A. Partition Quadratic Cost	5
2.1	Restatement	5
2.2	Algorithm and proof	5
2.3	Complexity	5
2.4	Full C++ solution	5
2.5	Full Python solution	5
3	B. Window Minimum DP	7
3.1	Restatement	7
3.2	Algorithm and proof	7
3.3	Complexity	7
3.4	Full C++ solution	7
3.5	Full Python solution	7
4	C. Optimal Merge Knuth	8
4.1	Restatement	8
4.2	Algorithm and proof	8
4.3	Complexity	8
4.4	Full C++ solution	8
4.5	Full Python solution	8
5	D. Bounded-Length Maximum Sum	10
5.1	Restatement and constraints	10
5.2	Reduction and trace	10
5.3	Monotone deque algorithm	10
5.4	Correctness proof	10
5.5	Complexity and implementation notes	11
5.6	Full C++ solution	11
5.7	Full Python solution	12
5.8	Tests that matter	12
6	Testing discussion	13

1 Editorial overview

1.1 Problem-specific proof map

A. Partition Quadratic Cost. The transition chooses the previous cut; the cost matrix's monotone optima justify divide-and-conquer search over the candidate interval.

B. Window Minimum DP. At step i , expired indices cannot transition to i ; a monotonic deque retains exactly unexpired candidates in increasing DP value, so its front is the required minimum.

C. Optimal Merge Knuth. The interval recurrence tries the final split. Knuth's monotonicity bounds the optimal split for $[l, r]$ between neighboring optimal splits, preserving the same answer while reducing candidates.

1.2 Detailed solution guide

A. Divide-and-Conquer DP Shape. The transition has the form $dp_cur[i] = \min_{k < i} (dp_prev[k] + cost(k, i))$. When the optimal k values are monotone, divide-and-conquer computes the middle i , searches only the candidate range inherited from the parent call, and recurses left/right with tighter ranges. The local implementation may keep the direct transition for readability, but the editorial proof identifies exactly where monotonicity is used in the optimized version.

B. Monotone Queue DP. When a transition asks for the best value over a sliding window, store candidate indices in a deque. Remove indices that leave the window from the front. Before pushing a new index, remove worse candidates from the back. The front is always the best legal index because every earlier worse candidate has either expired or been dominated.

C. Knuth Optimization Shape. For interval DP $dp[l][r] = \min_k dp[l][k] + dp[k][r] + cost(l, r)$, Knuth optimization is valid when optima satisfy $opt[l][r-1] \leq opt[l][r] \leq opt[l+1][r]$. Then the search range for k is much smaller than the full interval. The local solution keeps the recurrence visible; the optimized version replaces the full scan by that bounded scan.

Common pitfalls. Never apply an optimization just because a recurrence looks similar. State the monotonicity or quadrangle inequality condition first, then implement the restricted search range.

This section is about reducing candidate transitions without changing the DP meaning.

- A uses partition DP with quadratic group cost. The optimized version relies on monotone optimal split points; the local reference may keep the transition explicit for readability.
- B is the canonical deque minimum DP: remove expired indices from the front and dominated candidates from the back.
- C is adjacent interval merging. Knuth optimization is valid for the classic sum-cost merge because optimal split points are monotone.

The editorial proof obligation is always the same: show that discarded candidates can never be optimal for any future state they would have served.

2 A. Partition Quadratic Cost

2.1 Restatement

Partition the array into k contiguous groups minimizing the sum of squared group sums.

2.2 Algorithm and proof

The task-specific reduction in the proof map maps legal original choices to exactly the generated states, and the final operation reads the requested optimum or classification. $\square$

2.3 Complexity

The bound is the concrete one stated for this task in the proof map; the code performs only its enumerations, searches, or transitions.

2.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; const long long INF=4e18; int main(){ios::sync_with_stdio(false);cin.tie(0);
```

2.5 Full Python solution

```
import sys
INF=10**30
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,k=data[0],data[1]; a=data[2:]; ps=[0]
    for x in a: ps.append(ps[-1]+x)
    def cost(l,r): return (ps[r]-ps[l])**2
    prev=[INF]*(n+1); prev[0]=0
    for _ in range(k):
        cur=[INF]*(n+1)
        def solve(lo,hi,optl,opttr):
            if lo>hi: return
            mid=(lo+hi)//2; best=(INF,optl)
```

```

        for j in range(optl,min(optr,mid-1)+1):
            val=prev[j]+cost(j,mid)
            if val<best[0]: best=(val,j)
            cur[mid]=best[0]; solve(lo,mid-1,optl,best[1]); solve(mid+1,hi,best[1],optr)
        solve(1,n,0,n-1); prev=cur
    print(prev[n])
if __name__=="__main__": main()

```

3 B. Window Minimum DP

3.1 Restatement

$dp[i] = a[i] + \min(dp[j])$ over the previous w positions. Print $dp[n]$.

3.2 Algorithm and proof

The task-specific reduction in the proof map maps legal original choices to exactly the generated states, and the final operation reads the requested optimum or classification. $\square$

3.3 Complexity

The bound is the concrete one stated for this task in the proof map; the code performs only its enumerations, searches, or transitions.

3.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,w;if(!(cin>>n>>w))return 0;vector<int> a(n);for(int i=0;i<n;i++)cin>>a[i];vector<int> dp(n+1,0);deque<int> dq;dq.push_back(0);for(int i=1;i<=n;i++){while(dq.size()>0&&dp[dq.back()]>=dp[i-1])dq.pop_back();dq.push_back(i-1);dp[i]=a[i]+dp[dq.front()];if(dq.front()==0)dp[i]=a[i];}
```

3.5 Full Python solution

```
import sys,collections
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,w=data[0],data[1]; a=data[2:]; dp=[0]*(n+1); dq=collections.deque([0])
    for i in range(1,n+1):
        while dq and dq[0]<i-w: dq.popleft()
        dp[i]=dp[dq[0]]+a[i-1]
        while dq and dp[dq[-1]]>=dp[i]: dq.pop()
        dq.append(i)
    print(dp[n])
if __name__=="__main__": main()
```

4 C. Optimal Merge Knuth

4.1 Restatement

Merge adjacent piles with cost equal to merged sum; print minimum total cost.

4.2 Algorithm and proof

The task-specific reduction in the proof map maps legal original choices to exactly the generated states, and the final operation reads the requested optimum or classification. $\square$

4.3 Complexity

The bound is the concrete one stated for this task in the proof map; the code performs only its enumerations, searches, or transitions.

4.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;if(!(cin>
```

4.5 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n=data[0]; a=data[1:]; ps=[0]
    for x in a: ps.append(ps[-1]+x)
    dp=[[0]*n for _ in range(n)]; opt=[[0]*n for _ in range(n)]
    for i in range(n): opt[i][i]=i
    for length in range(2,n+1):
        for l in range(n-length+1):
            r=l+length-1; best=10**30; bestk=l
            for k in range(opt[l][r-1], opt[l+1][r]+1):
                val=dp[l][k]+(dp[k+1][r] if k+1<=r else 0)+ps[r+1]-ps[l]
                if val<best: best=val; bestk=k
```

```
        dp[l][r]=best; opt[l][r]=bestk
    print(dp[0][n-1] if n else 0)
if __name__=="__main__": main()
```

5 D. Bounded-Length Maximum Sum

5.1 Restatement and constraints

Choose a contiguous subarray whose length is in $[L, R]$ and maximize its sum. Values may be negative, so an empty subarray is not permitted and initializing the answer to zero would be wrong. With n up to 200000, checking every start and end is quadratic.

5.2 Reduction and trace

Let $\text{pref}[i]$ be the sum of the first i values. A subarray ending just before prefix position right and beginning after prefix position left has sum $\text{pref}[\text{right}] - \text{pref}[\text{left}]$. Its length is allowed exactly when $\text{right} - R \leq \text{left} \leq \text{right} - L$.

For a fixed right , $\text{pref}[\text{right}]$ is fixed, so the best allowed subarray uses the smallest prefix sum in that sliding interval of left indices. With $L=2$, $R=4$, and values $-2, 3, -1, 5, -6, 4$, endpoint $\text{right}=4$ permits prefix indices $0, 1, 2$, whose sums are $0, -2, 1$. Choose index 1: the result is $\text{pref}[4] - \text{pref}[1] = 5 - (-2) = 7$, from subarray $3, -1, 5$.

5.3 Monotone deque algorithm

Keep a deque of legal prefix indices in increasing pref order. At each endpoint right , insert $\text{enter} = \text{right} - L$, removing suffix indices with $\text{pref}[\text{index}] \geq \text{pref}[\text{enter}]$; remove front indices below $\text{right} - R$; then update the answer with $\text{pref}[\text{right}] - \text{pref}[\text{deque.front}()]$. The entering index has just reached the minimum allowed length. A removed back index has a prefix sum no smaller than a newer index and expires earlier, so it can never be useful.

5.4 Correctness proof

Lemma 1. Before evaluating an endpoint right , the deque contains exactly the undominated prefix indices in $[\text{right} - R, \text{right} - L]$, in increasing prefix-sum order.

Proof. Index $\text{right} - L$ is inserted when it becomes legal and indices below $\text{right} - R$ are removed when they expire. If an older index has prefix sum at least a newer index's sum, the newer index is at least as good for every future endpoint and remains legal longer. Thus deleting the older index is safe; no other legal, undominated index is deleted.

Lemma 2. At endpoint right , the deque front gives the best allowed subarray ending there.

Proof. Lemma 1 makes the front the smallest legal $\text{pref}[\text{left}]$. Subtracting it from fixed $\text{pref}[\text{right}]$ maximizes $\text{pref}[\text{right}] - \text{pref}[\text{left}]$.

Theorem. The algorithm prints the maximum sum of every allowed subarray.

Proof. Every allowed subarray has an endpoint **right**. Lemma 2 considers the best allowed one for that endpoint, and the algorithm takes the maximum over every endpoint.

5.5 Complexity and implementation notes

Every prefix index enters and leaves the deque at most once, so time is $O(n)$ and memory is $O(n)$ for the prefix array and deque. Use `long long` in C++: the answer can have magnitude $2 * 10^{14}$. Insert before querying so length exactly L is legal. Removing equal prefix sums from the back retains the newer index, which expires later.

5.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, minimum_length, maximum_length;
    if (!(cin >> n >> minimum_length >> maximum_length)) return 0;
    vector<long long> prefix(n + 1);
    for (int index = 1; index <= n; ++index) {
        long long value;
        cin >> value;
        prefix[index] = prefix[index - 1] + value;
    }

    deque<int> candidates;
    long long answer = numeric_limits<long long>::lowest();
    for (int right = 1; right <= n; ++right) {
        int entering = right - minimum_length;
        if (entering >= 0) {
            while (!candidates.empty() && prefix[candidates.back()] >= prefix[entering]) {
                candidates.pop_back();
            }
            candidates.push_back(entering);
        }
        while (!candidates.empty() && candidates.front() < right - maximum_length) {
            candidates.pop_front();
        }
        if (!candidates.empty()) {
            answer = max(answer, prefix[right] - prefix[candidates.front()]);
        }
    }
    cout << answer << '\n';
}
```

5.7 Full Python solution

```
import sys
from collections import deque

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, minimum_length, maximum_length = data[:3]
    values = data[3:]
    prefix = [0] * (n + 1)
    for index, value in enumerate(values, 1):
        prefix[index] = prefix[index - 1] + value

    candidates: deque[int] = deque()
    answer = -10**30
    for right in range(1, n + 1):
        entering = right - minimum_length
        if entering >= 0:
            while candidates and prefix[candidates[-1]] >= prefix[entering]:
                candidates.pop()
            candidates.append(entering)
        while candidates and candidates[0] < right - maximum_length:
            candidates.popleft()
        if candidates:
            answer = max(answer, prefix[right] - prefix[candidates[0]])

    print(answer)

if __name__ == "__main__":
    main()
```

5.8 Tests that matter

```
All values negative: the answer must still be nonempty.
L = R = 1: this becomes the maximum element.
L = 1 and R = n: every nonempty length is legal.
Optima at exactly L and exactly R, plus equal prefix sums.
```

The tiny oracle enumerates every subarray independently. The scale case has 200000 ones and a wide length interval, exposing a quadratic scan.

6 Testing discussion

For A, the input contract requires nonnegative values. This makes the squared segment-sum cost Monge, so the optimal previous cut is monotone and the divide-and-conquer search restriction is valid. Do not reuse this optimization unchanged for signed arrays: the monotone-optimum theorem can fail even though the unoptimized recurrence remains correct.

Keep a slow recurrence for random arrays. Compare every DP entry, not only the final answer: an incorrect opt-range often first corrupts an interior state. For window minima, test $w=1$, $w \geq n$, and equal values; for Knuth, test one and two piles and verify interval iteration order.

Random tests compare C++ and Python against fixed expected outputs generated from the reference implementation. Important cases include singletons, negative numbers where allowed, repeated values, and boundary query sizes.